TrendCast: Fashion Demand Forecasting Using Machine Learning and Domain Adaptation

B SHILPA
Roll No. 23K91A0547
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

B HARI NARAYANA
Roll No. 23K91A0544
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

A RANI THEJASWINI
Roll No. 23K91A0516
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

A SANJANA
Roll No. 23K91A0511
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

DEEPTHI
[DESIGNATION] (project guide)
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

***Abstract*---**A fashion retailer must forecast demand for thousands of short-lived articles, and a model trained in one sales context often loses accuracy in another. This paper presents TrendCast, a web application that turns retail transactions into daily units sold per article, forecasts them with XGBoost from 31 calendar, price, attribute, lag and rolling features, and offers importance-weighted domain adaptation, in which a logistic domain classifier supplies a weight p/(1-p) for every source-domain row. We evaluate it on a real subset of the H&M retail data: 300 best-selling articles, 684,392 transactions and 107 days. On a chronological test window XGBoost reaches RMSE 11.09 against 12.89 for a moving average, but not against repeating yesterday's value (9.22). As shipped, adaptation lowers the target error on 2 of 6 source-target pairs; the one clear gain disappears when source rows that overlap the test dates are removed, and unnormalised weights can make the model collapse to a constant. We report these findings with controls, a one-line repair, and the measured speed of the software.

***Index Terms*---**covariate shift, demand forecasting, domain adaptation, fashion retail, importance weighting, XGBoost.

I. INTRODUCTION

Fashion retail is driven by short product life cycles, seasons and promotions. Planners usually set stock from last year's sales and personal judgement; an over-forecast leaves unsold stock, an under-forecast loses the sale. Gradient-boosted trees are now the usual learner for such tabular demand data [1], [2], and the base paper of this work adds layered explanations so that planners can trust them [1]. Two difficulties remain. A model fitted on one market, channel or product group is then used on another whose data are distributed differently, and its accuracy falls; and new articles have no history at all. Domain adaptation, which moves what was learned in a source domain to a shifted target domain [3], is the natural remedy, but it is rarely reported with controls that show whether it really helped.

Today a merchandiser typically combines a spreadsheet of last season's sales, a seasonal index and judgement, and then corrects the plan by hand when sales diverge. This is cheap but slow, it cannot use price, colour, department or promotion effects together, and it carries no measure of its own error. Machine-learning forecasts remove part of that work, yet they raise their own questions that planners ask first: is the model better than the naive rule they already use, will it still work in the next channel or season, and what does it get wrong? TrendCast is built so that these questions can be answered from its own screens and scripts.

This paper makes four contributions. (1) TrendCast, a tested pipeline and web console (FastAPI, React) from raw H&M-style files to forecasts, with a gate before every step. (2) An honest comparison of XGBoost with simple and tree baselines on real data, including a feature-group ablation and a rolling-origin backtest. (3) A six-pair study of importance-weighted adaptation with five seeds and four controls (target-only model, pooled data, shuffled weights, time-clean source), which exposes two defects of the shipped method. (4) Offline measurements the product itself does not show: exact TreeSHAP attribution, inventory-unit impact and unseen-article error. Section II reviews related work, Section III describes the method, Section IV gives the results and Section V concludes.

A. System Hardware

All numbers were measured on one machine: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Windows 11 Pro, no GPU, with other jobs running at the same time.

B. Software Requirements

Python 3.11.9, scikit-learn 1.5.0 [4], XGBoost 2.0.3 [5], LightGBM 4.7.0 [6] (baseline only), pandas 2.2.1, NumPy 1.26.4, FastAPI 0.143.3; the console uses React 19, Vite 8 and Recharts. Tests use pytest and Playwright; the optional AI explanation calls the Gemini REST service.

II. LITERATURE SURVEY

Base paper. Mishra et al. [1] forecast one-day-ahead SKU demand on UCI Online Retail II (1.07 million transactions, 4,631 SKUs) with LightGBM and 41 engineered features, tuned by Optuna (30 trials, three time-series folds), and compare it with naive lag-1, a 7-day moving average, linear regression, random forest [7], XGBoost and CatBoost. LightGBM reaches RMSE 33.71 and R2 0.243; CatBoost (33.83, p = 0.917) and XGBoost (33.89) are statistically close, and MASE is 0.799. Their contribution is explanation: permutation importance, partial dependence, SHAP [8], [9] and ICE curves, with rolling

statistics the dominant feature group (removing them changes R2 by -0.064). They state that validation uses a single dataset, that relationships are assumed stationary, that external factors are excluded and that one step ahead limits planning. Domain shift is not addressed.

Boosting and competitions. XGBoost [5] and LightGBM [6] made boosted trees the default for tabular data, and in the M5 retail competition boosted trees with hand-built features matched or beat deep models [2]. **Covariate shift.** When training and test inputs have different distributions, weighting each training row by the density ratio p_target(x)/p_source(x) restores consistency [10]; a discriminative source-versus-target classifier estimates that ratio without density estimation [11], importance-weighted validation selects models under shift [12], and a learning bound ties target error to a domain divergence [13]. Neural variants make features domain-invariant [14]. **Fashion.** Thomassey [15] reviews clothing sales forecasting, where volatile demand, seasonality and short-lived items dominate; recent work forecasts new fashion items from attributes [16], images and attributes with attention [17], or images with Google Trends [18].

TABLE I
RELATED WORK COMPARED

Work	Approach	Data	Reported result	Limitation
Mishra et al. [1]	LightGBM + PFI, PDP, SHAP, ICE	UCI Online Retail II, 4,631 SKUs	RMSE 33.71, R2 0.243	One dataset; no shift
Bickel et al. [11]	Logistic-classifier covariate-shift weights	Benchmark tasks	No density estimate needed	Needs overlap
Shimodaira [10]	Density-ratio weighted likelihood	Theory, simulation	Consistent if mis-specified	Large weights
Ekambaram et al. [17]	Attention, images + attributes	Large fashion data	Beats k-NN baselines	Needs images, history
Singh et al. [16]	Attribute models, new items	Fashion sales	Finds demand drivers	No market transfer
Skenderi et al. [18]	Images + Google Trends	VISUELLE, 5,577 items	Whole-horizon forecast	External signals

Gap. The surveyed work either applies gradient boosting with explanations to one retail market, or builds neural models for new fashion items. A gradient-boosted demand model moved to a differently distributed domain by a simple, inspectable adaptation step, and then checked against honest controls, is not reported. TrendCast implements that step and tests it.

III. METHODOLOGY

A. Architecture and Implementation

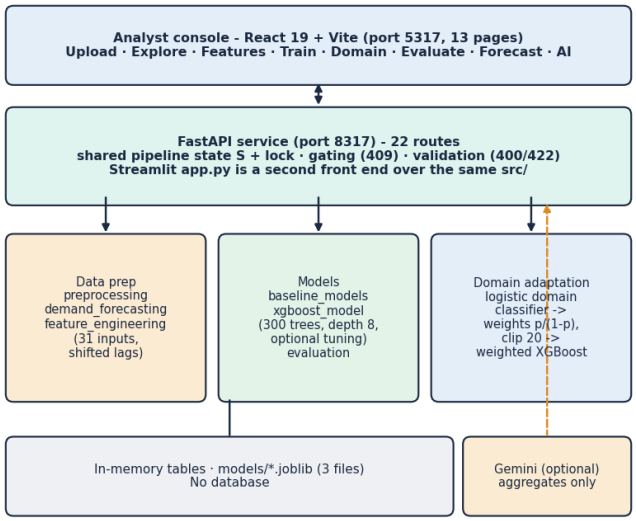


Fig. 1. TrendCast architecture: React console, FastAPI service with 22 routes over the src/ package, in-memory state and three model files; there is no database.

After an upload the console shows a data-quality report (rows read and removed, missing values, duplicates, date range, customers and products), then exploration charts of daily, weekly and monthly demand, the ten best-selling articles and demand by department, colour and channel, so that the analyst sees what cleaning removed before any model is fitted. The FastAPI service keeps one pipeline state behind a lock. Each route calls the same src/ functions as the original Streamlit page; later steps answer HTTP 409 with a plain message until earlier ones are done, bad files or parameters return 400 or 422, and the Gemini route returns 502 or 503 when the service is unreachable. The React console has 13 pages. The 7, 14 or 30-day forecast is recursive: each predicted value is appended to the history that builds the next day's lag and rolling features. The code is checked by 39 pytest tests (22 unit, 17 API; all pass) and a 26-check end-to-end smoke test (all pass).

B. Data and Preparation

We use the H&M Personalized Fashion Recommendations data [19]: transactions (date, customer, article, price, sales channel 1 or 2), articles (type, group, colour, department, section, garment group) and customers. The first transaction shard (2018-09-20 to 2019-01-04) was restricted to its 300 best-selling articles, giving 684,392 rows and 276,168 customers. Cleaning parses dates, drops rows with missing ids or non-positive price, and removes exact duplicate rows (79,214 rows, 11.6 %); the files code the channels only as 1 and 2. Rows are then left-joined to the article table and counted per article and day, which gives the target y = units sold. Missing days between an article's first and last sale are filled with zero, producing 31,094 article-days (2.0 % zero).

C. Features

The 31 model inputs are price, sales channel, six label-encoded attributes, fourteen calendar features and nine lag and rolling features. Month m and weekday d are also encoded cyclically,

$$x_{sin} = sin(2\pi t/T), \quad x_{cos} = cos(2\pi t/T) \qquad (1)$$

with T = 12 and 7. Lags are y(t-k) for k = 1, 7, 14, 28; rolling means (7, 14, 28 days) and standard deviations (7, 28) are computed on y shifted by one day, so the current day never enters its own features. The split is chronological: the first 80 % of dates train, the last 20 % test.

D. Model and Baselines

XGBoost [5] fits the regularised objective sum l(y, ŷ) + Σ Ω(f_k) with squared error, 300 trees, depth 8, learning rate 0.05, subsample and column sampling 0.8. Optional tuning is a randomised search [20] of 10 draws over six parameters with three time-ordered folds. Baselines are a 7-day moving average (the product's own), repeating yesterday, random forest, and LightGBM. Metrics are RMSE, MAE, R2, MAPE on non-zero days (zero days are counted and excluded) and WAPE = Σ|y-ŷ| / Σy.

$$RMSE = sqrt(\ mean\ (y - \hat{y})^2\ ), \quad WAPE = \Sigma|y - \hat{y}| / \Sigma y, \quad safe\ MAPE = mean_\{y>0\}\ |y - \hat{y}| / y \qquad (2)$$

Plain MAPE is undefined on days with no sales, which are common for single articles, so the product reports the average over days with y > 0 and states how many rows were left out. We add WAPE because it weights each article-day by its volume and so stays stable when many days are small, and R2 because it compares the model with always predicting the mean of the test window; a negative R2 therefore means worse than that constant.

E. Importance-Weighted Domain Adaptation

Source rows come from one value of a domain column (sales channel, product group or department), target rows from another; the column itself is removed from the inputs. A source-only model gives the before-adaptation error on the later half of the target dates. A logistic regression trained to separate source from target rows returns p = P(target | x), and each source row receives

$$w(x) = min(\ p(x) / (1 - p(x)),\ 20\ ) \qquad (3)$$

which equals the density ratio p_target/p_source up to a constant [10], [11]. XGBoost is refitted with these sample weights and scored on the same target rows. We add controls: a target-only model trained on the first half of the target dates, source plus target-train pooled without weights, shuffled weights, weights rescaled to mean 1, and a time-clean variant that drops source rows dated after the start of the target test window.

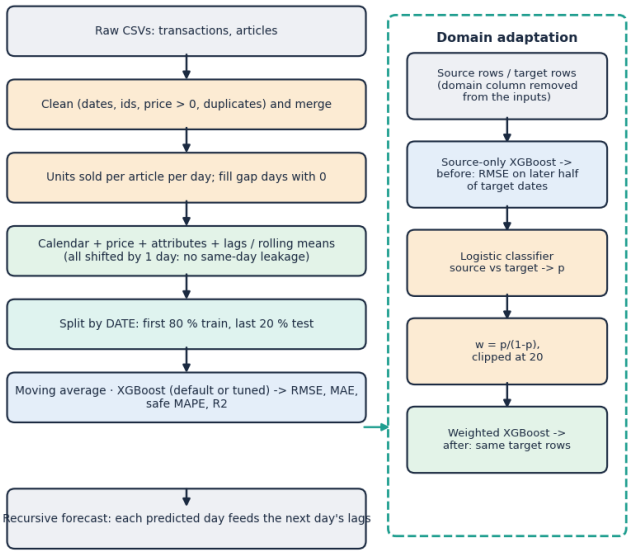


Fig. 2. Method: cleaning to features and the date split (left), and the domain-adaptation procedure (right); forecasts are made recursively.

F. Evaluation Protocol and Reproducibility

Every number comes from scripted experiments that write JSON files. Forecast accuracy uses the single chronological split and a three-fold rolling-origin backtest; XGBoost is repeated with five random seeds (42-46) because row and column sampling make it stochastic. Domain pairs are scored on the later half of the target dates only, so no target test row is ever used for training or for estimating weights except through the unlabelled inputs of the domain classifier, which sees target rows without their demand. The multi-day forecast is scored on the 30 days after an origin at which the model was trained, against the demand actually observed. Offline extras are SHAP values from XGBoost's built-in exact tree routine (they sum to the prediction to within 1e-4), error expressed as under- and over-forecast units, and a five-fold group split by article for unseen articles. The experiments, the tests and the screenshot capture can be rerun with one command each; the LightGBM baseline needs the library to be imported before pandas on this machine, which is noted in the installation guide.

IV. RESULTS

A. Test Conditions and Forecast Accuracy

The test window has 6515 article-days from 2018-12-14 to 2019-01-04, which contains the Christmas peak and the January sales, while the training window (2018-09-20 to 2018-12-13) holds no earlier year. XGBoost seeds vary little (RMSE 11.089 ± 0.048, five seeds). Table II shows that XGBoost beats the product's moving average (RMSE 11.09 against 12.89) but not repeating yesterday (9.22) or the shifted 7-day rolling mean (9.31); tuning (31 s) and LightGBM close part of the gap (10.34 and 10.06).

TABLE II
TEST-WINDOW ACCURACY, REAL H&M SUBSET

Model	RMSE	MAE	R2	WAPE %
Naive (lag 1)	9.22	5.65	0.540	45.6
Rolling mean 7 d	9.31	5.65	0.531	45.6
LightGBM	10.06	6.71	0.452	54.1
XGBoost (tuned)	10.34	6.88	0.421	55.6
XGBoost (default, 5 seeds)	11.09	7.08	0.335	57.1
Moving average (7 d)	12.89	8.09	0.101	65.3
Random forest	13.26	7.53	0.049	60.8

Mean absolute and percentage errors look large because demand is small and noisy: the mean article-day sells 19.5 units and MAPE, computed on non-zero days, is 143 % for XGBoost. Summed over all articles per day the picture is different: daily-total WAPE is 31.7 % for XGBoost and 24.7 % for the moving average, so the moving average tracks the aggregate level better even though XGBoost is more accurate article by article. Figure 3 shows the cause: XGBoost follows the rise to the pre-Christmas peak, but on 25 December and 1 January it predicts 4,367 and 6,393 units when 1,329 and 1,456 were sold. No input flags holidays (the calendar encodes dates, not public holidays, and a lag feature cannot foresee a one-day drop), a gap that matters in fashion retail. The tuned model uses depth 12, 400 trees and a learning rate of 0.01 and is still not better than repeating yesterday.

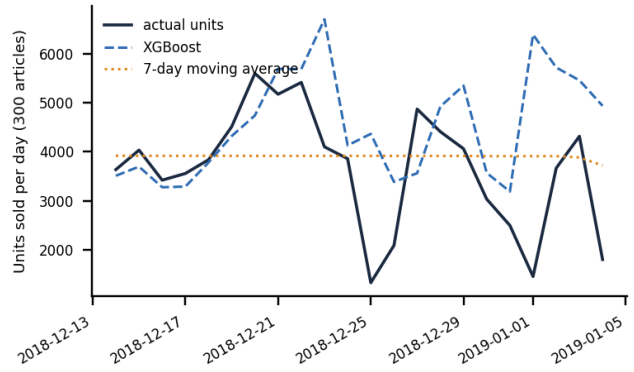


Fig. 3. Daily units sold over the test window (sum of the 300 articles) with the XGBoost and moving-average forecasts; XGBoost follows the pre-Christmas rise but not the drops on 25 December and 1 January.

The ablation explains why. Without lag and rolling features RMSE rises to 16.16 (R2 -0.41); calendar features alone give 15.18 (R2 -0.25). A model using only the nine lag and rolling features scores 9.40 (R2 0.52) and beats the shipped 31-feature model (11.06), because absolute-time inputs such as quarter and day of year cannot extrapolate beyond the training dates. In a three-fold rolling-origin backtest XGBoost beats the moving average in every fold (for example 9.63 against 12.44) but beats repeating yesterday only in the first (27.65 against 30.10); this agrees with the base paper's finding that rolling statistics dominate.

B. Recursive Multi-Day Forecast

From the origin 2018-12-06, over 296 articles, the app's recursive forecast has WAPE 41 %, 77 % and 124 % on days 1-7, 8-14 and 15-30, against 34 %, 48 % and 79 % for a flat mean of the last seven days. Refreshing the calendar features that the loop leaves frozen did not help. The multi-day feature should therefore be read as indicative only.

C. Domain Adaptation

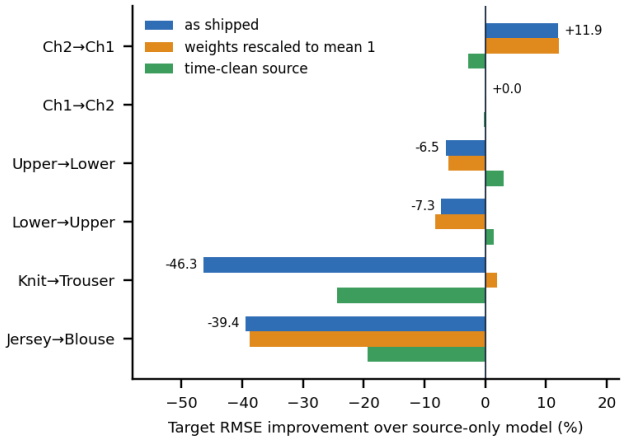


Fig. 4. Change in target RMSE from adaptation for six source-target pairs (positive is better), as shipped and with two repairs; Ch1, Ch2 are sales channels.

Averaged over five seeds, adaptation as shipped lowers target RMSE on 2 of 6 pairs (mean -14.6 %, range -46.3 % to +11.9 %). The only clear gain is channel 2 to channel 1: RMSE 10.80 to 9.50 (+11.9 %, better in 5 of 5 seeds). It is not an adaptation effect: with the time-clean source it becomes -2.8 %, and shuffled weights already give +2.2 %. Channel 1 to channel 2 changes by +0.01 %, and the product-group pairs worsen by 6.5 % and 7.3 %.

The department pairs fail badly (-46.3 % for knitwear to trousers). There the domain classifier separates the domains perfectly (accuracy 100 %), so p/(1-p) is tiny for every source row (mean weight 5.1e-05); XGBoost's minimum child weight of 1 can then never be met, the trees stop splitting and the model predicts a constant (R2 -0.01). Rescaling the same weights to mean 1 removes the collapse (+1.9 %), and the mean change over all pairs improves from -14.6 % to -6.5 %. Under the time-clean protocol the mean is -7.1 %, and with both repairs +0.2 %: adaptation is neutral. Simply pooling the source with the first half of the target, without weights, beats weighted adaptation on five of six pairs, and a target-only model is worse than the source-only model on five of six, because half of a small target is too little data.

TABLE III
DOMAIN ADAPTATION, TARGET RMSE (MEAN OF 5 SEEDS)

Pair	Clf acc %	Source-only	Adapted	Pooled	Target-only
Ch2→Ch1	67	10.80	9.50	9.78	10.05
Ch1→Ch2	66	19.00	18.99	17.84	22.46
Upper→Lower	93	13.90	14.80	13.51	18.21
Lower→Upper	93	11.91	12.78	12.13	16.49
Knit→Trouser	100	17.23	25.21	16.88	21.79
Jersey→Blouse	100	12.20	16.99	11.54	13.55

Table III also shows why the pairs behave differently. The channel pairs are only weakly separable (classifier accuracy 66 %), so the weights stay in a moderate range and adaptation changes little. Product groups and departments are

separated almost perfectly by the encoded attributes that remain in the inputs, which leaves no overlap between the domains: for upper body to lower body only 4 % of the rows carry effective weight and 0.1 % are clipped at 20. Importance weighting needs overlap between source and target inputs [10], [13]; where the domains are close to disjoint, a model cannot be moved by re-weighting rows, and the honest remedy is target data, as the pooled column shows. The shuffled-weights control is the safeguard against over-reading small gains: it gives the weights the same distribution but attaches them to random rows, so any improvement it also shows is noise or a side effect of re-weighting rather than of the learned density ratio. Each XGBoost run is stable (RMSE standard deviation across seeds up to 0.52), so the differences in the table are not seed noise except on the smallest target.

D. Explanations, Inventory Impact and New Articles

The app shows gain importance (top features is_month_end, lag_1, quarter). Exact TreeSHAP values, computed offline with XGBoost's own routine, rank lag_1 first (mean |SHAP| 8.7 units) and agree only moderately with gain (Spearman 0.40): gain over-rates calendar features that split often but contribute little. On the test window XGBoost over-forecasts total demand by 23.6 % yet under-forecasts 16.4 % of units sold and over-forecasts 39.9 %; the moving average under-forecasts 29.4 % and over-forecasts 35.9 %. For articles never seen in training, attributes and calendar alone give R2 0.27 (product-type mean -0.01), against 0.68 when the article's own history is known; the product itself has no cold-start mode.

E. Output Screens and Speed



Fig. 5. TrendCast domain-adaptation screen on the real subset: shift histogram and box plot of the most shifted feature, and the before-and-after cards (channel 1 to channel 2).

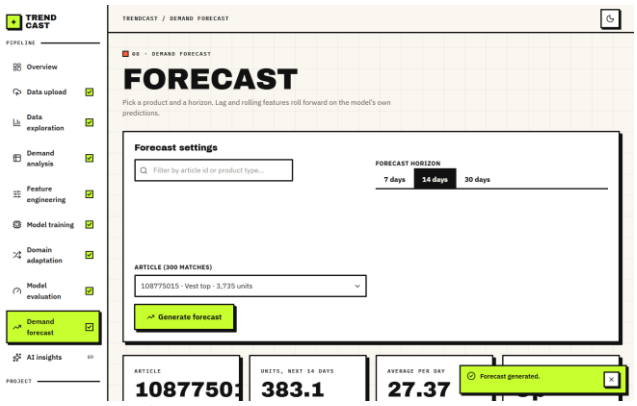


Fig. 6. TrendCast forecast screen: 60 days of history and a 14-day recursive forecast for one article, with the forecast table below.

On the 684,392-row subset, upload with cleaning takes 3.5 s, building features 0.9 s, default XGBoost training 1.1 s, tuning 30 s and a domain run 3.7 s; a 30-day forecast has a median latency of 156 ms.

F. Comparison with the Base Paper

The base paper and this work use different data (UCI Online Retail II, a UK wholesaler, against H&M fashion sales), different aggregation and metric scales, so absolute figures cannot be compared. Relative findings can: the base paper's LightGBM is 7.8 % better in RMSE than its moving average (33.71 against 36.57) and 29.4 % better than naive lag-1 (47.77), whereas XGBoost here is 14.0 % better than the moving average but 20.2 % worse than lag-1. Both studies find lag and rolling features dominant (base paper: R2 change of -0.064 on removal; here R2 falls from 0.34 to -0.41). Our R2 (0.335) is higher than the base paper's 0.243 only because the series, window and target differ.

G. Failure Cases and Observed Limits

The window is 107 days of 300 best-selling articles, so no yearly seasonality can be learned and the test period is atypical. Cleaning removes repeat purchases that are real demand. The sales-channel and price of the target day are model inputs and are frozen during recursive forecasts; the domain label is the day's most frequent channel although demand counts both channels. The domain classifier does not converge on unscaled features. The product does not show SHAP or inventory impact, and the AI explanation depends on an external service that answered 503 or timed out when the screens were captured. No user study, load test or full-size data run was done. Two further threats to validity apply to every number above. The 300 articles are the best sellers of one shard, so they sell on most days and are easier to forecast than the long tail of articles that sell rarely; and the test window is a single three-week period, so a different window, such as a quiet month, could change the ranking of the models, although the three backtest folds point the same way. The comparison with the base paper is therefore a comparison of findings, not of scores.

V. CONCLUSION AND FUTURE SCOPE

TrendCast delivers a complete, tested path from retail transactions to forecasts and an inspectable domain-adaptation experiment. Against its objectives, the forecasting model beats the product's own baseline (RMSE 11.09 against 12.89) but not repeating yesterday; explanations are partly met (gain importance in the app, SHAP offline); error is translated to inventory units only offline. The central finding is negative and useful: as shipped, importance-weighted adaptation helps on 2 of 6 pairs, its one clear gain comes from time overlap, and unnormalised weights can collapse the model; with repairs it is neutral, and plain pooling does better. The limits are the short 107-day window, one data shard, mixed-channel domains and no new-product mode.

Practical use follows from the findings. A planner who adopts TrendCast should treat the repeat-yesterday and rolling-mean forecasts as the bar to beat, keep the multi-day forecast for orientation only, check any adaptation against a pooled model and a time-clean split before trusting it, and read the AI explanation as a summary of the shown numbers, not as new evidence.

Future work: normalise weights and use the time-clean protocol by default; refresh calendar features in recursive forecasts and add direct multi-horizon and probabilistic models; drop absolute-time inputs or add last-year seasonality from the full two-year data; add a cold-start model from attributes and images; show SHAP and inventory cost in the console; and test other adaptation methods such as adversarial or correlation alignment [14] on regional or store domains.

REFERENCES

[1] L. N. Mishra, B. Senapati, S. K. Nayak and A. J. Rangari, "Explainable retail demand forecasting: A hybrid LightGBM-SHAP framework for inventory optimization," IEEE Access, vol. 14, pp. 43613-43633, 2026, doi: 10.1109/ACCESS.2026.3675540.

[2] S. Makridakis, E. Spiliotis and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," International Journal of Forecasting, vol. 38, no. 4, pp. 1346-1364, Oct. 2022.

[3] S. J. Pan and Q. Yang, "A survey on transfer learning," IEEE Transactions on Knowledge and Data Engineering, vol. 22, no. 10, pp. 1345-1359, Oct. 2010.

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, Aug. 2016, pp. 785-794.

[6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Proc. Advances in Neural Information Processing Systems (NIPS), 2017, pp. 3146-3154.

[7] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[8] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. Advances in Neural Information Processing Systems (NIPS), 2017, pp. 4765-4774.

[9] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nature Machine Intelligence, vol. 2, no. 1, pp. 56-67, Jan. 2020.

[10] H. Shimodaira, "Improving predictive inference under covariate shift by weighting the log-likelihood function," Journal of Statistical Planning and Inference, vol. 90, no. 2, pp. 227-244, 2000.

[11] S. Bickel, M. Brueckner and T. Scheffer, "Discriminative learning under covariate shift," Journal of Machine Learning Research, vol. 10, pp. 2137-2155, 2009.

[12] M. Sugiyama, M. Krauledat and K.-R. Mueller, "Covariate shift adaptation by importance weighted cross validation," Journal of Machine Learning Research, vol. 8, pp. 985-1005, 2007.

[13] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira and J. W. Vaughan, "A theory of learning from different domains," Machine Learning, vol. 79, no. 1-2, pp. 151-175, 2010.

[14] Y. Ganin et al., "Domain-adversarial training of neural networks," Journal of Machine Learning Research, vol. 17, no. 59, pp. 1-35, 2016.

[15] S. Thomassey, "Sales forecasts in clothing industry: The key success factor of the supply chain management," International Journal of Production Economics, vol. 128, no. 2, pp. 470-483, Dec. 2010.

[16] P. K. Singh, Y. Gupta, N. Jha and A. Rajan, "Fashion retail: Forecasting demand for new items," arXiv:1907.01960, 2019.

[17] V. Ekambaram, K. Manglik, S. Mukherjee, S. S. K. Sajja, S. Dwivedi and V. Raykar, "Attention based multi-modal new product sales time-series forecasting," in Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2020, pp. 3110-3118, doi: 10.1145/3394486.3403362.

[18] G. Skenderi, C. Joppi, M. Denitto and M. Cristani, "Well Googled is half done: Multimodal forecasting of new fashion product sales with image-based Google Trends," arXiv:2109.09824, 2021 (rev. 2024).

[19] H&M Group, "H&M Personalized Fashion Recommendations," Kaggle competition data (articles, customers, transactions), 2022. [Online]. Available: https://www.kaggle.com/competitions/h-and-m-personalized-fashion-recommendations

[20] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," Journal of Machine Learning Research, vol. 13, pp. 281-305, 2012.